Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №2

з дисципліни
«Об’єктно-орієнтоване програмування»
на тему
«Розробка графічного редактора об’єктів на C#»

Виконала:

студентка групи ІМ-52
Кондратюк Олександра Вікторівна
Номер в списку - 8

Перевірив:

Рекечинський Д.О.

Київ 2026

Мета роботи:

Отримати вміння та навички використовувати інкапсуляцію, абстракцію типів, успадкування та поліморфізм на основі класів, запрограмувавши простий графічний редактор в об’єктно-орієнтованому стилі.

Завдання:

1. Створити у середовищі розробки новий проєкт з ім’ям Lab2.
2. Скомпілювати проєкт і отримати виконуваний файл програми.
3. Перевірити роботу програми. За потреби налагодити програму.
4. Проаналізувати та прокоментувати результати та вихідний текст програми.
5. Оформити звіт.

Завдання згідно варіанту:

Номер студента в журналі Ж = 8. Меню «Об’єкти» розташоване між меню «Файл» і «Довідка» та містить чотири підпункти: «Крапка», «Лінія», «Прямокутник», «Еліпс». Параметри варіанту:

Параметр	**Значення для Ж = 8**
Масив об’єктів	8 mod 3 = 2 ≠ 0 → статичний масив; N = Ж + 100 = 108
«Гумовий» слід	8 mod 4 = 0 → суцільна лінія чорного кольору
Ввід прямокутника	8 mod 2 = 0 → по двох протилежних кутах
Відображення прямокутника	8 mod 5 = 3 → чорний контур без заповнення
Ввід еліпса	8 mod 2 = 0 → від центру до одного з кутів охоплюючого прямокутника
Відображення еліпса	8 mod 5 = 3 → чорний контур з кольоровим заповненням

Колір заповнення еліпса	8 mod 6 = 2 → світло-зелений
Позначка поточного типу об’єкта	8 mod 2 = 0 → у меню (аналог OnInitMenuPopup)

Вихідні тексти програми

Головний файл (Form1.cs)

```
using System;
using System.Drawing;
using System.Windows.Forms;
using Lab2.Shapes;

namespace Lab2
{
   public partial class Form1 : Form
   {
      private Shape[] pcshape = new Shape[108];
      private int shapeCount = 0;

      private bool isDrawing = false;
      private int startX, startY, currentX, currentY;

      private int currentShapeType = 1;

      public Form1()
      {
         InitializeComponent();
         this.DoubleBuffered = true;
         menuLine.Checked = true;
      }

      private void menuPoint_Click(object sender, EventArgs e) { SetShapeType(0,
menuPoint); }
      private void menuLine_Click(object sender, EventArgs e) { SetShapeType(1,
menuLine); }
      private void menuRect_Click(object sender, EventArgs e) { SetShapeType(2,
menuRect); }
      private void menuEllipse_Click(object sender, EventArgs e) { SetShapeType(3,
menuEllipse); }

      private void SetShapeType(int type, ToolStripMenuItem clickedMenu)
      {
         currentShapeType = type;
```

```
    menuPoint.Checked = false;
    menuLine.Checked = false;
    menuRect.Checked = false;
    menuEllipse.Checked = false;
    clickedMenu.Checked = true;
}

private void Form1_MouseDown(object sender, MouseEventArgs e)
{
    if (shapeCount >= 108) return;

    isDrawing = true;
    startX = e.X;
    startY = e.Y;
    currentX = e.X;
    currentY = e.Y;
}

private void Form1_MouseMove(object sender, MouseEventArgs e)
{
    if (isDrawing)
    {
        currentX = e.X;
        currentY = e.Y;
        this.Invalidate();
    }
}

private void Form1_MouseUp(object sender, MouseEventArgs e)
{
    if (!isDrawing) return;
    isDrawing = false;

    Shape newShape = null;

    switch (currentShapeType)
    {
        case 0: newShape = new PointShape(); break;
        case 1: newShape = new LineShape(); break;
        case 2: newShape = new RectShape(); break;
        case 3: newShape = new EllipseShape(); break;
    }

    if (newShape != null)
    {
```

```
            newShape.x1 = startX;
            newShape.y1 = startY;
            newShape.x2 = e.X;
            newShape.y2 = e.Y;

            pcshape[shapeCount] = newShape;
            shapeCount++;
        }

        this.Invalidate();
    }

    private void Form1_Paint(object sender, PaintEventArgs e)
    {
        for (int i = 0; i < shapeCount; i++)
        {
            if (pcshape[i] != null)
            {
                pcshape[i].Show(e.Graphics);
            }
        }

        if (isDrawing)
        {
            Pen rubberPen = new Pen(Color.Black, 1);

            switch (currentShapeType)
            {
                case 0:
                    e.Graphics.FillEllipse(Brushes.Black, currentX - 2, currentY - 2, 4, 4);
                    break;
                case 1:
                    e.Graphics.DrawLine(rubberPen, startX, startY, currentX, currentY);
                    break;
                case 2:
                    int rx = Math.Min(startX, currentX);
                    int ry = Math.Min(startY, currentY);
                    e.Graphics.DrawRectangle(rubberPen, rx, ry, Math.Abs(currentX -
startX), Math.Abs(currentY - startY));
                    break;
                case 3:
                    int rx_e = Math.Abs(currentX - startX);
                    int ry_e = Math.Abs(currentY - startY);
                    e.Graphics.DrawEllipse(rubberPen, startX - rx_e, startY - ry_e, 2 *
rx_e, 2 * ry_e);
```

```
                    break;
                }
            }
        }
    }
}
```

Файл Shapes

Shape.cs

```
using System.Drawing;

namespace Lab2.Shapes
{
    public abstract class Shape
    {
        public int x1, y1, x2, y2;

        public abstract void Show(Graphics g);
    }

}
```

Rectangle.cs

```
using System;
using System.Drawing;

namespace Lab2.Shapes
{
    public class RectShape : Shape
    {
        public override void Show(Graphics g)
        {
            Pen pen = new Pen(Color.Black, 1);

            int rectX = Math.Min(x1, x2);
            int rectY = Math.Min(y1, y2);
            int width = Math.Abs(x2 - x1);
            int height = Math.Abs(y2 - y1);

            g.DrawRectangle(pen, rectX, rectY, width, height);
        }
    }
```

```
}
```

Point.cs

```
using System.Drawing;

namespace Lab2.Shapes
{
   public class PointShape : Shape
   {
      public override void Show(Graphics g)
      {
         g.FillEllipse(Brushes.Black, x1 - 2, y1 - 2, 4, 4);
      }
   }

}
```

Line.cs

```
using System.Drawing;

namespace Lab2.Shapes
{
   public class LineShape : Shape
   {
      public override void Show(Graphics g)
      {
         g.DrawLine(Pens.Black, x1, y1, x2, y2);
      }
   }

}
```

Ellipse.cs

```
using System;
using System.Drawing;

namespace Lab2.Shapes
{
   public class EllipseShape : Shape
   {
      public override void Show(Graphics g)
      {
         Pen pen = new Pen(Color.Black, 1);
         Brush brush = new SolidBrush(Color.LightGreen);
```

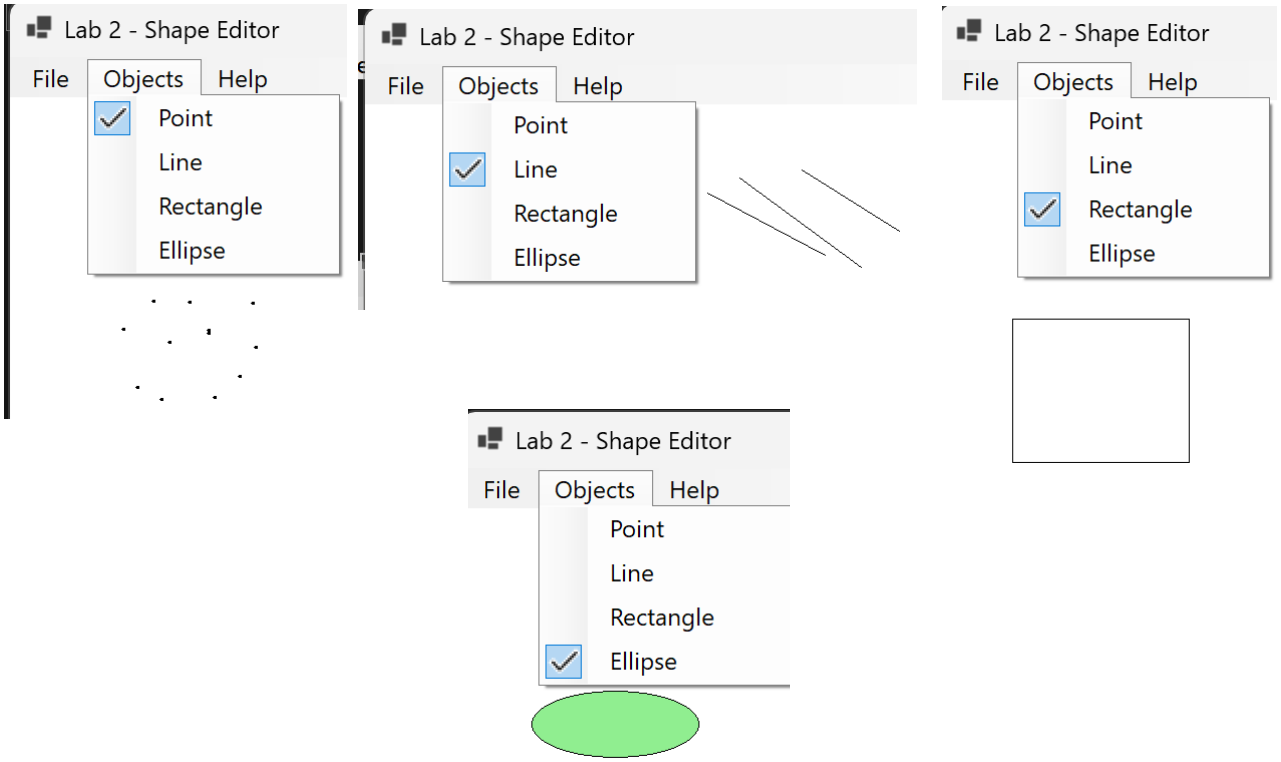
```
            int rx = Math.Abs(x2 - x1);
            int ry = Math.Abs(y2 - y1);

            int rectX = x1 - rx;
            int rectY = y1 - ry;
            int width = 2 * rx;
            int height = 2 * ry;

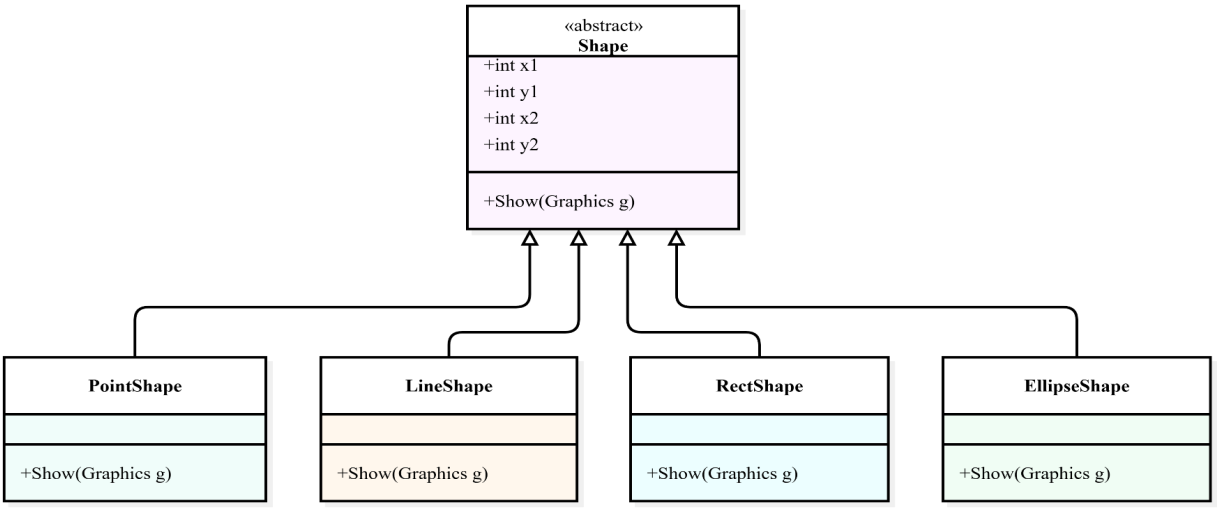
            g.FillEllipse(brush, rectX, rectY, width, height);
            g.DrawEllipse(pen, rectX, rectY, width, height);
        }
    }

}
```

Скриншоти роботи програми:



Діаграма класів



Висновки:

Під час виконання лабараторної роботи я отримала навички використовування інкапсуляції, абстракції типів, успадкування та поліморфізму на основі класів, запрограмувавши простий графічний редактор в об’єктно-орієнтованому стилі.